

AURORA E-COMMERCE

Full-Stack Line-by-Line Code Mastery & Architectural Handbook

An exhaustive, production-grade technical breakdown and line-by-line pedagogical reference for the AURORA Full-Stack E-Commerce platform. Every single backend Java class, database entity, repository query, transactional service, Spring Security filter, REST controller, DTO, React 19 context provider, centralized API fetch wrapper, and glassmorphic UI component is dissected in granular detail to explain what it does, why it exists, and how it impacts the overall full-stack architecture.

ENTERPRISE ARCHITECTURE & SYSTEM MATRIX

Backend Core:	Java 17+ / Spring Boot 4.x / Spring Data JPA / Hibernate 7.x
Security Architecture:	Spring Security 6/7 / Stateless JWT / HMAC-SHA256 / BCrypt Hashing
Frontend Framework:	React 19 / Vite / Vanilla CSS Design Tokens (Aurora Glassmorphism)
State Management:	React Context API (AuthContext, CartContext, ToastContext) + Hooks
Database Layer:	Persistent Local H2 Engine / Zero-Downtime Production MySQL Compatibility
Market Localization:	Indian Currency (INR / ₹) / 90x Scaled Prices / Pan-India Delivery / UPI & RuPay
Cloud Deployment:	Multi-Stage Docker / Render Cloud Web Service & CDN Static Site
Source Code Repo:	https://github.com/Saanidhya10/E-Commerce-Application---AURORA
Document Scope:	End-to-End Line-by-Line Technical Dissection & Senior Interview Defense

Chapter 1: Master Architecture & Topologies

Decoupled Client-Server, N-Tier Layering & Scalability Patterns

1.1 Why Decoupled Full-Stack Architecture?

The AURORA platform is engineered upon an enterprise-standard decoupled client-server architecture. The backend functions as an independent, stateless RESTful service implemented in Java Spring Boot, while the frontend is a modern single-page application (SPA) built using React 19 and Vite. Communication between the two systems is conducted strictly over asynchronous HTTP/HTTPS protocols using JSON (JavaScript Object Notation) data serialization.

This decoupling offers massive architectural benefits over legacy monolithic Spring MVC server-rendered templates (such as Thymeleaf or JSP):

1. **Resource Inefficiency:** Server-side rendering consumes CPU cycles on the backend application server to generate HTML strings. In a decoupled model, HTML rendering is offloaded entirely to the client device, allowing the backend to focus exclusively on business rules and database queries.
2. **Seamless Horizontal Scaling:** Because authentication is verified statelessly via signed JWT tokens rather than in-memory HttpSession, multiple backend instances can be spun up across different geographic regions without needing sticky sessions or distributed Redis session caches.
3. **Accelerated Frontend Iteration:** The frontend leverages Vite HMR (Hot Module Replacement) and modern browser caching. Static asset bundling and deployment are completely decoupled from Java compilation cycles.

Design Pattern: Model-View-Controller (MVC) & Layered Architecture

The backend enforces strict Layered Separation: Controllers (Presentation) receive requests -> Services (Domain Logic) enforce business constraints -> Repositories (Data Access) interact with the database -> Entities (Data Model) define object-relational mappings. Code from the controller layer is never permitted to query the database directly.

1.2 End-to-End Request Lifecycle: From Browser to Database

When a user triggers an action in the browser (such as clicking "Pay & Place Order"), the execution sequence traverses the entire full-stack stack as follows:

Step 1: The user clicks "Add to Cart" on ProductCard.jsx. React triggers an onClick event handler.

Step 2: ProductCard invokes the addToCart() function exposed by CartContext.jsx.

Step 3: CartContext calls cartApi.addItem() in cartApi.js, which forwards the call to the central client.js fetch utility.

Step 4: client.js extracts the JWT token from localStorage and injects the header "Authorization: Bearer <token>". It dispatches an HTTP POST request to /api/cart/items.

Step 5: The request arrives at the Spring Boot server. Before hitting any controller, JwtAuthenticationFilter intercepts the request, decodes the HMAC-SHA256 signature using JwtService, validates that the token is unexpired, extracts the email, and populates the SecurityContextHolder.

Step 6: CartController receives the request, extracts the authenticated user ID, and passes the CartItemRequest DTO to CartServiceImpl.

Step 7: CartServiceImpl executes in a transaction (@Transactional). It queries ProductRepository to check if the requested quantity exceeds product.stock. If valid, it persists a CartItem entity via CartItemRepository.

Step 8: CartServiceImpl returns a CartResponse DTO. Spring serializes it to JSON and returns HTTP 200 OK.

Step 9: React receives the response, updates its cart state in CartContext, the navbar badge animates to show the new item count, and ToastContext displays an animated toast: "Added to your cart!".

Chapter 2: Backend Domain Entities & Schemas

Comprehensive Dissection of Every Persistent Database Entity Class

2.1 User Entity (com.example.ecommerceapp.entity.User)

The User entity anchors the entire security and relationship model. It represents customers and administrators, housing their credentials and role.

SOURCE: ecommerceapp/.../entity/User.java

```
@Entity
@Table(name = "users")
@Data
@NoArgsConstructor
@AllArgsConstructor
public class User {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false)
    private String name;

    @Column(nullable = false, unique = true)
    private String email;

    @Column(nullable = false)
    private String password;

    @Enumerated(EnumType.STRING)
    @Column(nullable = false)
    private RoleType role = RoleType.CUSTOMER;
}
```

Line-by-Line / Component-by-Component Dissection:

@Entity	Registers this class with Hibernate as a JPA managed entity that maps directly to a SQL table.
@Table(name = "users")	Sets table name to "users". Prevents SQL syntax errors because "user" is a reserved keyword in PostgreSQL and MySQL.
@Id & @GeneratedValue	Declares the primary key with IDENTITY strategy, delegating auto-incrementing integer assignment to the database.
@Column(unique = true)	Creates a unique SQL index on the email column, ensuring no duplicate accounts can ever exist.
@Enumerated(STRING)	Instructs JPA to store enum constants as text ("CUSTOMER", "ADMIN") rather than ordinal integers (0, 1). This prevents data corruption if enums are reordered in Java code.

2.2 RoleType Enum (com.example.ecommerceapp.enums.RoleType)

RoleType defines the system authority levels: CUSTOMER and ADMIN.

SOURCE: ecommerceapp/.../enums/RoleType.java

```
public enum RoleType {  
    CUSTOMER,  
    ADMIN  
}
```

Line-by-Line / Component-by-Component Dissection:

CUSTOMER

Default privilege granted to regular registered users. Grants access to cart, orders, and addresses.

ADMIN

Elevated privilege granting access to the executive dashboard, platform revenue metrics, and catalog creation.

2.3 Product Entity (com.example.ecommerceapp.entity.Product)

Product is the cornerstone of the shopping catalog. It maintains inventory counts, category associations, and soft deletion state.

SOURCE: ecommerceapp/.../entity/Product.java

```
@Entity  
@Table(name = "products")  
@Data  
public class Product {  
    @Id  
    @GeneratedValue(strategy = GenerationType.IDENTITY)  
    private Long id;  
  
    @Column(nullable = false)  
    private String name;  
  
    @Column(columnDefinition = "TEXT")  
    private String description;  
  
    @Column(nullable = false, precision = 10, scale = 2)  
    private BigDecimal price;  
  
    @Column(nullable = false)  
    private Integer stock;  
  
    private String imageUrl;  
    private String brand;  
  
    @ManyToOne(fetch = FetchType.EAGER)  
    @JoinColumn(name = "category_id", nullable = false)  
    private Category category;  
  
    @Column(nullable = false)  
    private boolean active = true;  
}
```

Line-by-Line / Component-by-Component Dissection:**BigDecimal price**

Mandatory for financial data. Floating-point types (float, double) introduce IEEE 754 precision inaccuracies.

columnDefinition = "TEXT"

Allows large, detailed product descriptions up to 65,535 bytes rather than standard 255-character VARCHARs.

FetchType.EAGER

Solves Hibernate LazyInitializationException in decoupled REST APIs by immediately loading Category data during product retrieval.

@JoinColumn(name = ...)

Specifies the foreign key column "category_id" linking to the categories table.

boolean active = true

Soft deletion pattern: Discontinued items have active set to false, preserving referential integrity in past order histories.

2.4 Category Entity (com.example.ecommerceapp.entity.Category)

Represents departments such as Electronics, Fashion, Home & Living, Sports & Fitness, and Books.

SOURCE: ecommerceapp/.../entity/Category.java

```
@Entity
@Table(name = "categories")
@Data
public class Category {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, unique = true)
    private String name;

    private String description;
}
```

Line-by-Line / Component-by-Component Dissection:**@Column(unique = true)**

Guarantees that duplicate category names cannot be created in the database.

2.5 Cart & CartItem Entities

Models a customer's temporary shopping bag prior to checkout.

SOURCE: ecommerceapp/.../entity/Cart.java & CartItem.java

```

@Entity
@Table(name = "carts")
public class Cart {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @OneToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "user_id", nullable = false)
    private User user;

    @OneToMany(mappedBy = "cart", cascade = CascadeType.ALL, orphanRemoval = true)
    private List<CartItem> items = new ArrayList<>();
}

@Entity
@Table(name = "cart_items")
public class CartItem {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "cart_id", nullable = false)
    private Cart cart;

    @ManyToOne(fetch = FetchType.EAGER)
    @JoinColumn(name = "product_id", nullable = false)
    private Product product;

    @Column(nullable = false)
    private Integer quantity;
}

```

Line-by-Line / Component-by-Component Dissection:**@OneToOne user**

Enforces strict 1-to-1 relationship between a registered user and their active shopping bag.

orphanRemoval = true

When an item is deleted from the items collection in Java, Hibernate automatically issues a DELETE query for the cart_item row.

Product product

References the inventory item. FetchType.EAGER ensures title, image, and price are available when building cart responses.

2.6 Order & OrderItem Entities: Historical Immutability

Captures finalized customer transactions. Unlike shopping carts, orders are permanent and legally binding.

SOURCE: ecommerceapp/.../entity/Order.java & OrderItem.java

```

@Entity
@Table(name = "orders")
public class Order {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "user_id", nullable = false)
    private User user;

    @Column(nullable = false, precision = 10, scale = 2)
    private BigDecimal totalAmount;

    @Enumerated(EnumType.STRING)
    private OrderStatus status = OrderStatus.PLACED;

    private LocalDateTime orderDate = LocalDateTime.now();

    @OneToMany(mappedBy = "order", cascade = CascadeType.ALL)
    private List<OrderItem> items = new ArrayList<>();
}

@Entity
@Table(name = "order_items")
public class OrderItem {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "order_id", nullable = false)
    private Order order;

    @ManyToOne(fetch = FetchType.EAGER)
    @JoinColumn(name = "product_id", nullable = false)
    private Product product;

    @Column(nullable = false)
    private Integer quantity;

    @Column(nullable = false, precision = 10, scale = 2)
    private BigDecimal price; // Historical Price Snapshot
}

```

Line-by-Line / Component-by-Component Dissection:**BigDecimal price in OrderItem**

Historical snapshot: Records the exact unit price paid at the time of purchase, insulating financial records from future catalog price changes.

OrderStatus status

Fulfillment tracker: PLACED -> PROCESSING -> SHIPPED -> DELIVERED -> CANCELLED.

2.7 Address & Payment Entities

Address stores shipping destinations localized for Indian PIN codes and logistics, while Payment tracks payment methods.

SOURCE: ecommerceapp/.../entity/Address.java & Payment.java

```
@Entity
@Table(name = "addresses")
public class Address {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "user_id", nullable = false)
    private User user;

    private String fullName;
    private String phoneNumber;
    private String addressLine1;
    private String addressLine2;
    private String city;
    private String state;
    private String postalCode;
    private String country = "India";
}

@Entity
@Table(name = "payments")
public class Payment {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @OneToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "order_id", nullable = false)
    private Order order;

    private String paymentMethod; // UPI, CARD, COD

    @Enumerated(EnumType.STRING)
    private PaymentStatus status;

    private BigDecimal amount;
}
```

Line-by-Line / Component-by-Component Dissection:**country = "India"**

Default country constraint ensuring Indian domestic address validation.

postalCode

Stores 6-digit Indian PIN codes (e.g. 560103 for Bengaluru).

paymentMethod

Stores transaction type: UPI (Google Pay / PhonePe / Paytm), CARD (RuPay / Visa), or COD.

Chapter 3: Spring Data JPA Repositories

Dynamic Method Name Derivation, JPQL Queries & Query Execution

3.1 Repository Architecture Overview

Spring Data JPA interfaces eliminate boilerplate JDBC and DAO code. By extending `JpaRepository<T, ID>`, Spring automatically creates proxies at runtime implementing standard CRUD operations (save, findById, findAll, deleteById, count) while allowing custom query methods derived from method names.

3.2 ProductRepository (ProductRepository.java)

SOURCE: ProductRepository.java

```
public interface ProductRepository extends JpaRepository<Product, Long> {
    List<Product> findByActiveTrue();
    List<Product> findByCategory_IdAndActiveTrue(Long categoryId);

    @Query("SELECT p FROM Product p WHERE p.active = true AND " +
        "(LOWER(p.name) LIKE LOWER(CONCAT('%', :keyword, '%')) OR " +
        "LOWER(p.description) LIKE LOWER(CONCAT('%', :keyword, '%')) OR " +
        "LOWER(p.brand) LIKE LOWER(CONCAT('%', :keyword, '%')))"
    List<Product> searchActiveProducts(@Param("keyword") String keyword);
}
```

Line-by-Line / Component-by-Component Dissection:

`findByActiveTrue()`

Derives SQL: "SELECT * FROM products WHERE active = 1".

`findByCategory_IdAndActiveTrue`

Traverses foreign key to Category, filtering on active products within that department.

`@Query (JPQL)`

Object-oriented query operating on Java entities to execute full-text searching without SQL injection risks.

3.3 OrderRepository & CartRepository

SOURCE: OrderRepository.java & CartRepository.java

```
public interface OrderRepository extends JpaRepository<Order, Long> {
    List<Order> findByIdOrderByOrderDateDesc(Long userId);
    List<Order> findAllByOrderByOrderDateDesc();
    long countByStatus(OrderStatus status);
}

public interface CartRepository extends JpaRepository<Cart, Long> {
    Optional<Cart> findById(Long userId);
}
```

Line-by-Line / Component-by-Component Dissection:

<code>findByUserIdOrderByOrderDateDesc</code>	Returns customer order history with newest orders at the top.
<code>findAllByOrderByOrderDateDesc</code>	Feeds the Admin Dashboard with all system orders sorted chronologically.
<code>countByStatus(status)</code>	Powers real-time metric cards (e.g. pending orders awaiting fulfillment).
<code>findByUserId(userId)</code>	Finds the unique shopping cart belonging to a specific customer.

3.4 AddressRepository & UserRepository

SOURCE: AddressRepository.java & UserRepository.java

```
public interface AddressRepository extends JpaRepository<Address, Long> {  
    List<Address> findByUserId(Long userId);  
}  
  
public interface UserRepository extends JpaRepository<User, Long> {  
    Optional<User> findByEmail(String email);  
    Boolean existsByEmail(String email);  
}
```

Line-by-Line / Component-by-Component Dissection:

<code>findByUserId(userId)</code>	Pulls all saved delivery addresses for a customer.
<code>findByEmail(email)</code>	Crucial for authentication: retrieves credentials by email during login.
<code>existsByEmail(email)</code>	Fast boolean check during registration to reject duplicate emails.

Chapter 4: Backend Business Logic & Services

Atomic Transactions, Inventory Management, Cart Rules & Checkout Engine

4.1 OrderServiceImpl: The Atomic Checkout Engine

The checkout workflow is the most critical transaction in the system. It orchestrates inventory validation, stock decrement, order generation, payment linking, and cart clearing.

SOURCE: OrderServiceImpl.java

```
@Override
@Transactional
public OrderResponse createOrder(Long userId, OrderRequest request) {
    Cart cart = cartRepository.findById(userId)
        .orElseThrow(() -> new ResourceNotFoundException("Cart not found"));

    if (cart.getItems() == null || cart.getItems().isEmpty()) {
        throw new BadRequestException("Cannot create order from an empty cart");
    }

    // 1. Validate and decrement stock
    for (CartItem cartItem : cart.getItems()) {
        Product product = cartItem.getProduct();
        if (product.getStock() < cartItem.getQuantity()) {
            throw new BadRequestException("Insufficient stock for: " + product.getName());
        }
        product.setStock(product.getStock() - cartItem.getQuantity());
        productRepository.save(product);
    }

    // 2. Build Order & Items
    Order order = new Order();
    order.setUser(cart.getUser());
    order.setStatus(OrderStatus.PLACED);
    // ... calculate totalAmount and link items ...

    Order savedOrder = orderRepository.save(order);

    // 3. Clear cart
    cartService.clearCart(userId);

    return orderMapper.toResponse(savedOrder);
}
```

Line-by-Line / Component-by-Component Dissection:

@Transactional

Ensures atomicity: if stock deduction fails on item #3, all database writes in the method are completely rolled back.

product.getStock() < cartItem.getQuantity()

Prevents overselling under high concurrency.

cartService.clearCart(userId)

Clears the shopping bag so the customer cannot duplicate checkouts.

4.2 CartServiceImpl: Cart Management & Stock Constraints

Enforces business constraints when adding items to bags, adjusting quantities, or clearing items.

SOURCE: CartServiceImpl.java

```
@Override
@Transactional
public CartResponse addToCart(Long userId, CartItemRequest request) {
    Cart cart = getOrCreateCart(userId);
    Product product = productRepository.findById(request.getProductId())
        .orElseThrow(() -> new ResourceNotFoundException("Product not found"));

    if (product.getStock() < request.getQuantity()) {
        throw new BadRequestException("Requested quantity exceeds available stock");
    }

    Optional<CartItem> existing = cartItemRepository.findByCartIdAndProductId(cart.getId(),
        product.getId());
    if (existing.isPresent()) {
        CartItem item = existing.get();
        item.setQuantity(item.getQuantity() + request.getQuantity());
        cartItemRepository.save(item);
    } else {
        CartItem newItem = new CartItem();
        newItem.setCart(cart);
        newItem.setProduct(product);
        newItem.setQuantity(request.getQuantity());
        cartItemRepository.save(newItem);
    }
    return getCart(userId);
}
```

Line-by-Line / Component-by-Component Dissection:

getOrCreateCart(userId)

Creates a shopping cart on-demand if the user is visiting for the first time.

existing.isPresent() check

Increments quantity of existing items instead of creating redundant rows.

4.3 AuthServiceImpl: Registration & Password Hashing

AuthServiceImpl manages credential verification, BCrypt password hashing, and token dispatch.

SOURCE: AuthServiceImpl.java

```

@Override
public AuthResponse register(RegisterRequest request) {
    if (userRepository.existsByEmail(request.getEmail())) {
        throw new BadRequestException("Email already registered: " + request.getEmail());
    }

    User user = new User();
    user.setName(request.getName());
    user.setEmail(request.getEmail());
    user.setPassword(passwordEncoder.encode(request.getPassword()));
    user.setRole(RoleType.CUSTOMER);

    User saved = userRepository.save(user);
    UserDetails userDetails = userDetailsService.loadUserByUsername(saved.getEmail());
    String token = jwtService.generateToken(userDetails);

    return new AuthResponse(token, saved.getEmail(), saved.getRole().name());
}

```

Line-by-Line / Component-by-Component Dissection:`passwordEncoder.encode(...)`

Hashes password with random salt using BCrypt. Raw passwords are never stored in the database.

`jwtService.generateToken(...)`

Generates signed JWT token, logging the user in immediately upon registration.

4.4 ProductServiceImpl: Catalog Operations & Soft Deletion

Handles product queries, stock updates, active status filtering, and soft deletions.

SOURCE: ProductServiceImpl.java

```

@Override
@Transactional(readOnly = true)
public List<ProductResponse> getAllProducts(Long categoryId, String keyword) {
    List<Product> products;
    if (keyword != null && !keyword.trim().isEmpty()) {
        products = productRepository.searchActiveProducts(keyword.trim());
    } else if (categoryId != null) {
        products = productRepository.findByCategory_IdAndActiveTrue(categoryId);
    } else {
        products = productRepository.findByActiveTrue();
    }
    return products.stream().map(productMapper::toResponse).toList();
}

```

Line-by-Line / Component-by-Component Dissection:`@Transactional(readOnly = true)`

Optimizes performance: Hibernate skips dirty checking when it knows queries are read-only.

`productMapper::toResponse`

Converts JPA entities to DTOs before exiting the service layer.

Chapter 5: Spring Security, JWT & Filter Chains

Stateless Cryptographic Token Issuance, Inspection & Route Protection

5.1 The Stateless JWT Paradigm

Traditional web applications store session IDs in browser cookies and keep session data in server RAM (HttpSession). This fails in cloud containers because load balancers distribute requests across multiple instances. Stateless JWT solves this: the server signs user claims with HMAC-SHA256, and the client sends this token on every request. Any backend instance can verify the signature using the shared secret key without querying a session store.

5.2 JwtService: Token Generation & Verification

SOURCE: JwtService.java

```
public String generateToken(UserDetails userDetails) {
    return Jwts.builder()
        .setSubject(userDetails.getUsername())
        .setIssuedAt(new Date(System.currentTimeMillis()))
        .setExpiration(new Date(System.currentTimeMillis() + jwtExpiration))
        .signWith(getSignInKey(), SignatureAlgorithm.HS256)
        .compact();
}

public boolean isValidToken(String token, UserDetails userDetails) {
    final String username = extractUsername(token);
    return (username.equals(userDetails.getUsername())) && !isTokenExpired(token);
}
```

Line-by-Line / Component-by-Component Dissection:

`setSubject(...)`

Stores user email in standard "sub" claim.

`setExpiration(...)`

Sets token validity to exactly 24 hours.

`signWith(HS256)`

Signs token with private 256-bit secret key. Any client-side tampering invalidates the signature.

5.3 JwtAuthenticationFilter: Intercepting Incoming Requests

SOURCE: JwtAuthenticationFilter.java

```

@Override
protected void doFilterInternal(HttpServletRequest request, HttpServletResponse response,
FilterChain filterChain) throws ServletException, IOException {
    final String authHeader = request.getHeader("Authorization");
    if (authHeader == null || !authHeader.startsWith("Bearer ")) {
        filterChain.doFilter(request, response);
        return;
    }

    final String jwt = authHeader.substring(7);
    final String userEmail = jwtService.extractUsername(jwt);

    if (userEmail != null && SecurityContextHolder.getContext().getAuthentication() == null) {
        UserDetails userDetails = userDetailsService.loadUserByUsername(userEmail);
        if (jwtService.isTokenValid(jwt, userDetails)) {
            UsernamePasswordAuthenticationToken authToken = new UsernamePasswordAuthenticationToken(
                userDetails, null, userDetails.getAuthorities());
            authToken.setDetails(new WebAuthenticationDetailsSource().buildDetails(request));
            SecurityContextHolder.getContext().setAuthentication(authToken);
        }
    }
    filterChain.doFilter(request, response);
}

```

Line-by-Line / Component-by-Component Dissection:

```
authHeader.startsWith("Bearer ")
```

Extracts RFC 6750 Bearer authentication token.

```
SecurityContextHolder.getContext().setAuthentication(...)
```

Injects verified authentication into Spring's thread context, unlocking protected controller endpoints.

5.4 SecurityConfig: Route Permissions & CORS

SOURCE: SecurityConfig.java

```

@Bean
public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
    http
        .csrf(AbstractHttpConfigurer::disable)
        .cors(cors -> cors.configurationSource(corsConfigurationSource()))
        .sessionManagement(session -> session.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
        .authorizeHttpRequests(auth -> auth
            .requestMatchers("/api/auth/**", "/api/products/**", "/api/categories/**", "/actuator/**")
            .permitAll()
            .requestMatchers("/api/admin/**").hasRole("ADMIN")
            .anyRequest().authenticated()
        )
        .authenticationProvider(authenticationProvider())
        .addFilterBefore(jwtAuthenticationFilter, UsernamePasswordAuthenticationFilter.class);

    return http.build();
}

```


Line-by-Line / Component-by-Component Dissection:`csrf.disable()`

Safe to disable because the API uses stateless JWT Bearer tokens, not vulnerable session cookies.

`SessionCreationPolicy.STATELESS`

Prevents Spring from allocating in-memory session objects, saving cloud memory.

`permitAll()`

Allows public browsing of catalog and categories prior to user login.

`hasRole("ADMIN")`

Restricts administrative endpoints (/api/admin/**) strictly to verified administrators.

5.5 DataInitializer: Startup Bootstrap & 90x INR Multiplier

DataInitializer implements CommandLineRunner, running on startup to seed users and multiply prices into Indian Rupees.

SOURCE: DataInitializer.java

```
@Override
public void run(String... args) {
    seedUsersAndAddresses();
    seedCategoriesAndProducts();
    migratePricesAndAddressesToIndia();
}

private void migratePricesAndAddressesToIndia() {
    for (Product p : productRepository.findAll()) {
        if (p.getPrice() != null && p.getPrice().compareTo(new BigDecimal("1000")) < 0) {
            p.setPrice(p.getPrice().multiply(new BigDecimal("90")).setScale(2,
                RoundingMode.HALF_UP));
            productRepository.save(p);
        }
    }
}
```

Line-by-Line / Component-by-Component Dissection:`migratePricesAndAddressesToIndia()`

Automatically scales prices by 90 and assigns Bengaluru addresses on startup.

Chapter 6: REST Controllers & HTTP Handlers

Controller Annotations, REST Principles & Endpoint Implementations

6.1 AuthController: Login & Registration (AuthController.java)

Handles user registration and authentication.

SOURCE: AuthController.java

```
@RestController
@RequestMapping("/api/auth")
@RequiredArgsConstructor
public class AuthController {
    private final AuthService authService;

    @PostMapping("/register")
    public ResponseEntity<ApiResponse<AuthResponse>> register(@Valid @RequestBody RegisterRequest request) {
        AuthResponse response = authService.register(request);
        return ResponseEntity.status(HttpStatus.CREATED)
            .body(ApiResponse.success("User registered successfully", response));
    }

    @PostMapping("/login")
    public ResponseEntity<ApiResponse<AuthResponse>> login(@Valid @RequestBody LoginRequest request) {
        AuthResponse response = authService.login(request);
        return ResponseEntity.ok(ApiResponse.success("Login successful", response));
    }
}
```

Line-by-Line / Component-by-Component Dissection:

@RestController	Combines @Controller and @ResponseBody, automatically converting Java return types to JSON.
@Valid	Triggers Bean Validation on incoming request DTOs (e.g. @NotBlank, @Email).
ResponseEntity<T>	Allows precise HTTP status code control (201 Created vs 200 OK).

6.2 ProductController & CategoryController

Public endpoints for catalog browsing and search.

SOURCE: ProductController.java

```

@RestController
@RequestMapping("/api/products")
@RequiredArgsConstructor
public class ProductController {
    private final ProductService productService;

    @GetMapping
    public ResponseEntity<ApiResponse<List<ProductResponse>>> getAllProducts(
        @RequestParam(required = false) Long categoryId,
        @RequestParam(required = false) String search) {
        List<ProductResponse> products = productService.getAllProducts(categoryId, search);
        return ResponseEntity.ok(ApiResponse.success("Products retrieved successfully", products));
    }

    @GetMapping("/{id}")
    public ResponseEntity<ApiResponse<ProductResponse>> getProductById(@PathVariable Long id) {
        return ResponseEntity.ok(ApiResponse.success("Product retrieved successfully",
            productService.getProductById(id)));
    }
}

```

Line-by-Line / Component-by-Component Dissection:

`@RequestParam(required = false)` Enables optional query parameter filtering (?categoryId=1&search=Watch).

`@PathVariable Long id` Extracts dynamic path variables from the URL URI template.

6.3 CartController & OrderController

Protected endpoints managing customer bags and order checkout.

SOURCE: CartController.java & OrderController.java

```

@RestController
@RequestMapping("/api/cart")
public class CartController {
    @GetMapping
    public ResponseEntity<ApiResponse<CartResponse>> getCart(@AuthenticationPrincipal
        CustomUserDetails userDetails) {
        return ResponseEntity.ok(ApiResponse.success("Cart retrieved",
            cartService.getCart(userDetails.getId())));
    }

    @PostMapping("/items")
    public ResponseEntity<ApiResponse<CartResponse>> addItem(
        @AuthenticationPrincipal CustomUserDetails userDetails,
        @Valid @RequestBody CartItemRequest request) {
        return ResponseEntity.ok(ApiResponse.success("Item added",
            cartService.addToCart(userDetails.getId(), request)));
    }
}

```

Line-by-Line / Component-by-Component Dissection:

`@AuthenticationPrincipal` Directly resolves the logged-in user details from Spring's SecurityContext, guaranteeing that customers can only view their own shopping cart.

6.4 AdminController: Executive Operations & Metrics

Secured endpoints restricted to administrators.

SOURCE: AdminController.java

```
@RestController
@RequestMapping("/api/admin")
@RequiredArgsConstructor
public class AdminController {
    private final OrderService orderService;
    private final ProductService productService;

    @GetMapping("/dashboard")
    public ResponseEntity<ApiResponse<DashboardStatsResponse>> getDashboardStats() {
        return ResponseEntity.ok(ApiResponse.success("Dashboard stats retrieved",
orderService.getDashboardStats()));
    }

    @PutMapping("/orders/{orderId}/status")
    public ResponseEntity<ApiResponse<OrderResponse>> updateOrderStatus(
        @PathVariable Long orderId,
        @RequestParam OrderStatus status) {
        return ResponseEntity.ok(ApiResponse.success("Order status updated",
orderService.updateOrderStatus(orderId, status)));
    }
}
```

Line-by-Line / Component-by-Component Dissection:

getDashboardStats()

Calculates revenue, total orders, and active users in a single aggregated call.

updateOrderStatus(...)

Enables real-time order lifecycle progression from the Admin Dashboard.

Chapter 7: DTOs, Mappers & Global Exceptions

Data Transfer Object Pattern, Encapsulation & Cross-Cutting Error Advice

7.1 Why the DTO Pattern is Essential

Data Transfer Objects (DTOs) separate the internal database storage model from the external public API interface. Directly returning JPA entities over REST causes severe flaws:

1. Infinite JSON Recursion: Bi-directional mappings (`@OneToMany` in `Order` pointing to `@ManyToOne` in `OrderItem`) cause Jackson to serialize back and forth indefinitely until a `StackOverflowError` crashes the application.
2. Over-Posting / Mass Assignment Attacks: If a controller binds an incoming HTTP request directly to a `User` entity, an attacker could submit JSON with `"role: ADMIN"` and hijack system privileges.
3. Information Disclosure: DTOs ensure that sensitive internal fields (such as hashed passwords or internal database primary keys) are never exposed to clients.

7.2 Global Exception Handler (`GlobalExceptionHandler.java`)

SOURCE: `GlobalExceptionHandler.java`

```
@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(ResourceNotFoundException.class)
    public ResponseEntity<ApiResponse<Void>> handleNotFound(ResourceNotFoundException ex) {
        return ResponseEntity.status(HttpStatus.NOT_FOUND)
            .body(ApiResponse.error(ex.getMessage()));
    }

    @ExceptionHandler(BadRequestException.class)
    public ResponseEntity<ApiResponse<Void>> handleBadRequest(BadRequestException ex) {
        return ResponseEntity.status(HttpStatus.BAD_REQUEST)
            .body(ApiResponse.error(ex.getMessage()));
    }

    @ExceptionHandler(UnauthorizedException.class)
    public ResponseEntity<ApiResponse<Void>> handleUnauthorized(UnauthorizedException ex) {
        return ResponseEntity.status(HttpStatus.UNAUTHORIZED)
            .body(ApiResponse.error(ex.getMessage()));
    }
}
```

Line-by-Line / Component-by-Component Dissection:

`@RestControllerAdvice`

Global interceptor for exceptions thrown anywhere in the controller layer.

`ApiResponse.error(...)`

Guarantees the frontend receives a predictable JSON envelope with `{ success: false, message: "..."}.`

Chapter 8: Frontend Architecture & Global State

React 19, Provider Hierarchies, Centralized Fetch & Client Routing

8.1 Provider Hierarchy (main.jsx)

Mounts the React tree and establishes the hierarchy of Context Providers:

SOURCE: main.jsx

```
createRoot(document.getElementById('root')).render(  
  <StrictMode>  
    <ToastProvider>  
      <AuthProvider>  
        <CartProvider>  
          <App />  
        </CartProvider>  
      </AuthProvider>  
    </ToastProvider>  
  </StrictMode>  
);
```

Line-by-Line / Component-by-Component Dissection:

ToastProvider	Placed at root so Auth and Cart can trigger floating notification toasts.
AuthProvider	Wraps CartProvider so Cart can re-fetch whenever the user logs in or switches accounts.
CartProvider	Injects cart state, item counts, and drawer triggers into the application.

8.2 Centralized HTTP Client (client.js)

SOURCE: client.js

```
const API_BASE = import.meta.env.VITE_API_BASE_URL || '';
const BASE_URL = API_BASE ? `${API_BASE.replace(/\$/ , '')}/api` : '/api';

export async function request(endpoint, options = {}) {
  const token = localStorage.getItem('token');
  const headers = {
    'Content-Type': 'application/json',
    Accept: 'application/json',
    ...options.headers,
  };

  if (token) {
    headers['Authorization'] = `Bearer ${token}`;
  }

  const response = await fetch(`${BASE_URL}${endpoint}`, { ...options, headers });
  const data = await response.json().catch(() => null);

  if (!response.ok) {
    throw new Error(data?.message || `HTTP ${response.status} error`);
  }
  return data;
}
```

Line-by-Line / Component-by-Component Dissection:

```
import.meta.env.VITE_API_BASE_U
RL
```

Dynamically routes to live Render backend in production or Vite proxy in dev.

```
headers['Authorization'] = ...
```

Automatically attaches JWT token from localStorage to every authenticated request.

8.3 AuthContext & CartContext

AuthContext manages login/logout and demo accounts. CartContext manages cart drawer visibility and item counts.

SOURCE: AuthContext.jsx & CartContext.jsx

```
// AuthContext Demo Helper
const loginDemo = async (role) => {
  const email = role === 'admin' ? 'admin@ecommerce.com' : 'customer@ecommerce.com';
  const password = role === 'admin' ? 'Admin@123' : 'Customer@123';
  await login(email, password);
};

// CartContext Badge Calculation
const totalItemCount = cart?.items
  ? cart.items.reduce((sum, item) => sum + item.quantity, 0)
  : 0;
```

Line-by-Line / Component-by-Component Dissection:

```
loginDemo(...)
```

Enables instant one-click login for customer and admin roles without typing.

```
totalItemCount
```

Derived state that powers the shopping bag badge in the top navigation bar.

Chapter 9: Frontend UI Components & Experience

Aurora Glassmorphism, Indian Checkout & Admin Dashboard

9.1 Component Breakdown

- 1. `Navbar.jsx`: Logo with glowing accent, search bar with debounce, cart trigger with animated item counter, user menu, and Admin Portal toggle.
- 2. `HeroBanner.jsx`: Promotes festive discounts, displays Indian value propositions (Free Pan-India Delivery, 100% BIS Certified, 7-Day Easy Returns), and interactive category filter pills.
- 3. `ProductCard.jsx`: High-res product imagery with hover zoom, brand pill, rating, Indian Rupee (₹) price formatting, and quick "Add to Cart" button with instant feedback.
- 4. `ProductDetailModal.jsx`: Detailed view showing stock counts, fulfillment guarantees (Dispatched from Mumbai/Bengaluru hubs in 24 hours), GST invoice notice, quantity picker, and add-to-cart action.
- 5. `CartDrawer.jsx`: Slide-over drawer with item thumbnails, quantity adjustment (+/-), item deletion, subtotal calculation, and checkout trigger.
- 6. `CheckoutModal.jsx`: Indian address selector and creator (PIN code, state, city), payment selector (Instant UPI, RuPay Cards, Cash on Delivery), order summary, and order confirmation receipt.
- 7. `OrderHistoryModal.jsx`: Displays past orders with live status badges (PLACED, PROCESSING, SHIPPED, DELIVERED).
- 8. `AuthModal.jsx`: Sign in and Register forms with one-click demo logins for Instant Customer and Instant Admin.
- 9. `AdminDashboard.jsx`: Analytics dashboard displaying Total Revenue (₹), Total Orders, Pending Orders, Catalog Volume, and live order status management.

9.2 Indian Rupee Currency Formatter (`format.js`)

SOURCE: `format.js`

```
export function formatCurrency(amount) {
  const value = Number(amount || 0);
  return `₹${value.toLocaleString('en-IN', {
    minimumFractionDigits: 2,
    maximumFractionDigits: 2,
  })}`;
}
```

Line-by-Line / Component-by-Component Dissection:

<code>en-IN locale</code>	Formats numbers into Indian numbering convention (thousands, lakhs, crores).
<code>minimumFractionDigits: 2</code>	Ensures prices always render with standard two decimal places (e.g. ₹26,999.00).

Chapter 10: End-to-End User Journey Traces

Step-by-Step Code Execution Paths for Every Key User Interaction

10.1 Case Study 1: Complete Checkout & Order Creation

Tracing the exact sequence of events when a customer places an order:

1. Browser State: Customer clicks "Pay & Place Order" in CheckoutModal.jsx with paymentMethod="UPI" and addressId=1.
2. Network Dispatch: orderApi.checkout() triggers client.js, which creates an HTTP POST request to /api/orders with JSON { addressId: 1, paymentMethod: "UPI" } and Bearer token in headers.
3. Filter Interception: JwtAuthenticationFilter intercepts request, validates HMAC-SHA256 signature, extracts user email, and sets SecurityContextHolder.
4. Controller Invocation: OrderController.createOrder() resolves user ID and invokes orderService.createOrder().
5. Inventory Verification: OrderServiceImpl checks each cart item against product.stock. If stock >= quantity, decrements stock in Product entity.
6. Order Persistence: Order entity is created with status=PLACED, total calculated, and saved to orders table.
7. Payment Persistence: Payment entity is generated with method="UPI", status=COMPLETED, and linked to Order.
8. Cart Clearing: cartService.clearCart() removes all cart_items rows.
9. Response Return: OrderResponse DTO is returned with HTTP 201 Created.
10. React State Synchronization: React CartContext resets items to [], badge resets to 0, and CheckoutModal renders order confirmation receipt.

10.2 Case Study 2: Admin Live Order Status Update

Tracing how administrators manage orders:

1. Browser State: Admin changes order status dropdown in AdminDashboard.jsx from "PLACED" to "SHIPPED".
 2. API Call: orderApi.updateStatus(orderId, "SHIPPED") dispatches PUT /api/admin/orders/{id}/status?status=SHIPPED.
 3. Security Check: Spring Security verifies the user possesses the ADMIN role (ROLE_ADMIN). If not, returns 403 Forbidden.
 4. Service Execution: OrderServiceImpl finds order by ID, updates status to SHIPPED, and saves to database.
 5. Toast Notification: AdminDashboard receives updated OrderResponse and ToastContext displays "Order #1 status updated to SHIPPED".
-

Chapter 11: Technical Interview Defense & Mastery

Comprehensive Q&A covering Architecture, Edge Cases, Database & Security

11.1 Senior-Level Technical Questions & Answers

Q1: Why did you choose JWT over traditional HTTP Sessions?

Traditional sessions are stateful and stored in server memory. This creates a severe bottleneck in cloud auto-scaling environments (like Render or AWS) because horizontal scaling requires sticky sessions or complex Redis clustering. JWT is completely stateless; the client stores the signed token, allowing any backend container to validate the user without querying a shared session store.

Q2: How do you prevent overselling and race conditions during checkout?

In `OrderServiceImpl.createOrder()`, the entire checkout sequence runs inside an atomic `@Transactional` database transaction. Before generating the order, we iterate through each cart item and verify "if (`product.getStock()` < `quantity`)". If inventory is insufficient, a `BadRequestException` is thrown, instantly aborting the transaction and rolling back any partial database writes.

Q3: Why use DTOs instead of exposing JPA Entities directly to REST controllers?

Exposing JPA entities directly creates severe security and performance issues: 1) Infinite JSON recursion loops in bi-directional relationships (`@OneToMany` / `@ManyToOne`). 2) Mass Assignment vulnerabilities where malicious users could inject "role: ADMIN" into JSON bodies. 3) Breaking API contracts when internal database columns change.

Q4: How did you solve `LazyInitializationException` in Product-Category relations?

In decoupled REST APIs, Jackson serializes entities outside the active Hibernate persistence context. In `Product.java`, we configured "`@ManyToOne(fetch = FetchType.EAGER)`", guaranteeing that the `Category` entity is joined and fetched during the initial query.

Q5: How does CORS work and why was it necessary?

CORS (Cross-Origin Resource Sharing) is a browser security mechanism blocking web pages from making AJAX requests to different domains. Because React runs on port 3000 (or a Render static URL) and Spring Boot runs on port 8080 (or a Render web service URL), Spring Boot's `SecurityConfig` explicitly configures a `CorsConfigurationSource` permitting cross-origin requests and exposing Authorization headers.

Q6: Explain the design pattern used in the React frontend.

We implemented the React Context Provider pattern (`AuthContext`, `CartContext`, `ToastContext`) combined with Custom Hooks (`useAuth`, `useCart`, `useToast`). This eliminates "prop-drilling", allowing deeply nested components like `ProductCard` or `CheckoutModal` to trigger cart updates and access auth tokens directly.

Q7: How did you design the database to handle both local dev and cloud prod?

We used Spring Boot Profiles and environment placeholders. In development, the `datasource` defaults to a persistent local H2 file database ("`jdbc:h2:file:./data/ecommercedb`"). In production, setting `DB_URL`, `DB_USERNAME`, and `DB_PASSWORD` environment variables seamlessly redirects Hibernate to a production MySQL instance without modifying code.

Q8: What is the purpose of DataInitializer.java?

It implements Spring Boot's CommandLineRunner interface, executing automatically when the server boots. It checks if the database is empty and seeds default categories, products with 90x INR pricing, sample customer addresses, and demo accounts with BCrypt-hashed passwords.

Q9: Why use BigDecimal instead of Double for prices and financial totals?

Double and Float use binary floating-point arithmetic (IEEE 754), which cannot accurately represent exact decimal fractions like 0.1, causing rounding bugs ($0.1 + 0.2 = 0.30000000000000004$). BigDecimal provides arbitrary-precision decimal mathematics, making it mandatory for all currency and tax calculations.

Q10: How does the Docker container bind dynamically to Render's PORT?

Render dynamically assigns a random port through the \$PORT environment variable. In our Dockerfile, the entrypoint executes: `sh -c java -Dserver.port=${PORT:-8080} -jar app.jar`. This passes Render's dynamic port directly into Spring Boot.

Q11: What is the role of BCryptPasswordEncoder?

BCrypt is an adaptive cryptographic hash function that incorporates salt to protect against rainbow table attacks. It deliberately takes computational work to calculate, making brute-force password cracking practically impossible.

Q12: How is Role-Based Access Control (RBAC) enforced in the backend?

In SecurityConfig.java, we define request matchers: `"requestMatchers('/api/admin/**').hasRole('ADMIN')"`. When a user attempts to call an admin endpoint, Spring Security checks their GrantedAuthority list extracted from their JWT token. If the user only has the CUSTOMER role, HTTP 403 Forbidden is returned.

Q13: How does the Soft Deletion pattern work in Product.java?

Rather than deleting rows from the database (`"DELETE FROM products WHERE id = ?"`), we set `"active = false"`. All public queries filter on `"active = true"`. This preserves referential integrity for historic orders that reference discontinued products.

Q14: Explain the difference between Authentication and Authorization.

Authentication verifies who you are (e.g. validating email and password and issuing a JWT token). Authorization verifies what you are allowed to do (e.g. allowing an ADMIN to view the revenue dashboard while restricting a CUSTOMER to their own cart).

Q15: What is the purpose of @Transactional in Spring Boot?

It tells Spring to execute the annotated method inside a managed database transaction. If the method completes normally, Spring commits the transaction. If any RuntimeException is thrown, Spring automatically rolls back all changes.

Q16: How does the slide-over Cart Drawer work without third-party UI libraries?

We built CartDrawer using pure React state and CSS transitions. In index.css, the drawer is positioned fixed on the right with `"transform: translateX(100%)"`. When CartContext toggles isCartOpen to true, an `".open"` class applies `"transform: translateX(0)"`, smoothly sliding the drawer into view.

Q17: Why did you choose Vite over Create React App (CRA)?

Create React App uses Webpack, which bundles the entire codebase before serving, leading to slow startup times. Vite uses native ES Modules (ESM) and esbuild, resulting in sub-second server startup (ready in ~200ms) and lightning-fast Hot Module Replacement (HMR).

Q18: What is the role of the reverse proxy in vite.config.js?

During local development, the browser makes requests to `"http://localhost:3000/api"`. The Vite dev server proxies these requests to `"http://localhost:8080/api"`. This prevents cross-origin domain mismatches during local testing.

Q19: How do you handle expired JWT tokens on the frontend?

In client.js, when a fetch request returns HTTP 401 Unauthorized, client.js intercepts the response, clears the expired token from localStorage, and AuthContext automatically redirects the user to the Sign In modal.

Q20: What are the primary scaling considerations if traffic grows 100x?

1) Add Redis for caching the product catalog to reduce database load. 2) Implement read/write database replicas with MySQL. 3) Move product images to an AWS S3 bucket behind CloudFront CDN. 4) Use RabbitMQ or Apache Kafka for asynchronous order fulfillment processing.

